IoT Practicals 2–10

Wokwi project code and circuit configuration files

Prepared from the uploaded “iot-practical” PDF. Practical 1 is intentionally excluded, as requested.

File convention: Arduino C++ practicals include `sketch.ino`; Raspberry Pi Pico MicroPython practicals include `main.py`; each simulated practical includes a `diagram.json` circuit file.

> **Important:** Some original practicals require physical hardware or boards Wokwi does not emulate. Where needed, the document clearly labels a simulation substitute. Check the notes before using external cloud services.

Contents

Practical 2: Word and character count of a given string

Practical 3: Working principle of Raspberry Pi Pico with PIR interface

Practical 4: Bluetooth LED ON/OFF control using smartphone commands

Practical 5: Connect to available Wi-Fi using Arduino

Practical 6: Retrieve temperature and humidity data from ThingSpeak

Practical 7: Publish temperature and humidity to an MQTT broker

Practical 8: Interface a passive buzzer with Raspberry Pi Pico

Practical 9: IoT-based DC motor speed control using potentiometer and servo

Practical 10: IoT data logging to ThingSpeak using LDR

Practical 2 — Word and character count of a given string

Board / language: Raspberry Pi Pico — MicroPython

Based on Practical 2 in the supplied PDF.

diagram.json

```
{
  "version": 1,
  "author": "Wokwi",
  "editor": "wokwi",
  "parts": [
    {
      "type": "board-pi-pico",
      "id": "pico",
      "top": 0,
      "left": 0,
      "attrs": {}
    }
  ],
  "connections": []
}
```

main.py

```
# Practical 2: Word and character count
text = input("Enter a string: ")

word_count = len(text.split())
char_count = len(text)

print("Number of Words:", word_count)
print("Number of Characters:", char_count)
```

Wiring / compatibility note: Example from the PDF: “Welcome to Raspberry Pi” gives 4 words and 23 characters. If interactive input is unavailable in your simulator, temporarily replace the input line with text = "Welcome to Raspberry Pi".

Practical 3 — Working principle of Raspberry Pi Pico with PIR interface

Board / language: Raspberry Pi Pico — Arduino C++ (source practical uses Arduino code)

The uploaded PDF provides Arduino-style setup()/loop() code using GPIO 15 and Serial1.

diagram.json

```
{
  "version": 1,
  "author": "Wokwi",
  "editor": "wokwi",
  "parts": [
    {
      "type": "board-pi-pico",
      "id": "pico",
      "top": 0,
      "left": 0,
      "attrs": {}
    },
    {
      "type": "wokwi-pir-motion-sensor",
      "id": "pir1",
      "top": -30,
      "left": 180,
      "attrs": {}
    }
  ],
  "connections": [
    [
      "pir1:VCC",
      "pico:3V3(OUT)",
      "red",
      []
    ],
    [
      "pir1:GND",
      "pico:GND.1",
      "black",
      []
    ],
    [
      "pir1:OUT",
      "pico:GP15",
      "green",
      []
    ]
  ]
}
```

sketch.ino

```
// Practical 3: PIR motion detection on Raspberry Pi Pico
const int PIR_PIN = 15;

void setup() {
  pinMode(PIR_PIN, INPUT);
  Serial.begin(115200);
  Serial.println("Hello, Raspberry Pi Pico!");
}

void loop() {
  int pir = digitalRead(PIR_PIN);

  if (pir == HIGH) {
    Serial.println("MOVEMENT DETECTED");
  } else {
    Serial.println("NO MOVEMENT DETECTED");
  }

  delay(500);
}
```

Wiring / compatibility note: Wiring: PIR VCC → 3V3(OUT), GND → GND, OUT → GP15. The PDF uses Serial1; Serial is used here so the messages appear in Wokwi's Serial Monitor.

Practical 4 — Bluetooth LED ON/OFF control using smartphone commands

Board / language: Raspberry Pi Pico — MicroPython

Based on the PDF's MicroPython UART example and HC-05 wiring.

diagram.json

```
{
  "version": 1,
  "author": "Wokwi",
  "editor": "wokwi",
  "parts": [
    {
      "type": "board-pi-pico",
      "id": "pico",
      "top": 0,
      "left": 0,
      "attrs": {}
    },
    {
      "type": "wokwi-led",
      "id": "led1",
      "top": -20,
      "left": 190,
      "attrs": {
        "color": "green"
      }
    },
    {
      "type": "wokwi-resistor",
      "id": "r1",
      "top": 30,
      "left": 140,
      "attrs": {
        "value": "220"
      }
    }
  ],
  "connections": [
    [
      "pico:GP19",
      "r1:1",
      "green",
      []
    ],
    [
      "r1:2",
      "led1:A",
      "green",
      []
    ],
    [
      "led1:C",
      "pico:GND.1",
      "black",
      []
    ]
  ]
}
```

main.py

```
# Practical 4: LED control using serial commands
# Wokwi does not simulate HC-05 phone pairing.
# Use the serial terminal as a simulation substitute.
from machine import Pin

led = Pin(19, Pin.OUT)
print("Enter 1 or ON to turn LED on; 0 or OFF to turn it off.")

while True:
    command = input().strip().upper()
    if command in ("1", "ON"):
        led.value(1)
        print("LED ON")
    elif command in ("0", "OFF"):
        led.value(0)
        print("LED OFF")
    else:
        print("Invalid command. Enter 1/0 or ON/OFF.")
```

Wiring / compatibility note: The real practical requires an HC-05 Bluetooth module and a smartphone. Wokwi's simulation here demonstrates the LED logic through console input instead; GP19 drives the LED through a 220 Ω resistor.

Practical 5 — Connect to available Wi-Fi using Arduino

Board / language: ESP32 DevKit — Arduino C++

The PDF provides Arduino Wi-Fi code for Wokwi-GUEST.

diagram.json

```
{
  "version": 1,
  "author": "Wokwi",
  "editor": "wokwi",
  "parts": [
    {
      "type": "board-esp32-devkit-c-v4",
      "id": "esp",
      "top": 0,
      "left": 0,
      "attrs": {}
    }
  ],
  "connections": []
}
```

sketch.ino

```
#include <WiFi.h>

const char* ssid = "Wokwi-GUEST";
const char* password = "";

void setup() {
  Serial.begin(115200);
  WiFi.mode(WIFI_STA);
  WiFi.begin(ssid, password);

  Serial.print("Connecting to WiFi");
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }

  Serial.println("\\nConnected");
  Serial.print("WiFi status: ");
  Serial.println(WiFi.status());
  Serial.print("IP address: ");
  Serial.println(WiFi.localIP());
  Serial.print("RSSI: ");
  Serial.println(WiFi.RSSI());
}

void loop() {
  delay(1000);
}
```

Wiring / compatibility note: Run the simulation and open Serial Monitor at 115200 baud. Wokwi's simulated Wi-Fi network is Wokwi-GUEST with an empty password.

Practical 6 — Retrieve temperature and humidity data from ThingSpeak

Board / language: Raspberry Pi Pico — MicroPython

Preserves the uploaded PDF's MicroPython approach, public channel 12397, field4 temperature and field3 humidity.

diagram.json

```
{
  "version": 1,
  "author": "Wokwi",
  "editor": "wokwi",
  "parts": [
    {
      "type": "board-pi-pico",
      "id": "pico",
      "top": 0,
      "left": 0,
      "attrs": {}
    }
  ],
  "connections": []
}
```

main.py

```
import network
import urequests
import time

# Connect to Wokwi simulated Wi-Fi
wlan = network.WLAN(network.STA_IF)
wlan.active(True)
wlan.connect("Wokwi-GUEST", "")

while not wlan.isconnected():
    time.sleep(0.5)

print("WiFi connected")

# Public ThingSpeak demo channel from the practical PDF
CHANNEL_ID = "12397"
READ_KEY = ""
TEMP_FIELD = "field4"
HUM_FIELD = "field3"

url = "https://api.thingspeak.com/channels/" + CHANNEL_ID + "/feeds/last.json"
if READ_KEY:
    url += "?api_key=" + READ_KEY

while True:
    try:
        response = urequests.get(url)
        data = response.json()
        response.close()

        print("Time:", data.get("created_at"))
        print("Temp:", data.get(TEMP_FIELD))
        print("Humidity:", data.get(HUM_FIELD))
        print("-" * 30)
    except Exception as error:
        print("Network error:", error)

    time.sleep(20)
```

Wiring / compatibility note: This follows the PDF's channel ID and field mapping. Network access and the public channel's current fields must be available; some MicroPython firmware builds may need urequests support.

Practical 7 — Publish temperature and humidity to an MQTT broker

Board / language: Raspberry Pi Pico — MicroPython

Based on the PDF's MicroPython DHT22 + MQTTClient example.

diagram.json

```
{
  "version": 1,
  "author": "Wokwi",
  "editor": "wokwi",
  "parts": [
    {
      "type": "board-pi-pico",
      "id": "pico",
      "top": 0,
      "left": 0,
      "attrs": {}
    },
    {
      "type": "wokwi-dht22",
      "id": "dht1",
      "top": -30,
      "left": 180,
      "attrs": {
        "temperature": "24",
        "humidity": "40"
      }
    }
  ],
  "connections": [
    [
      "dht1:VCC",
      "pico:3V3(OUT)",
      "red",
      []
    ],
    [
      "dht1:GND",
      "pico:GND.1",
      "black",
      []
    ],
    [
      "dht1:SDA",
      "pico:GP15",
      "green",
      []
    ]
  ]
}
```

main.py

```
import network
import time
from machine import Pin
import dht
import ujson
from umqtt.simple import MQTTClient

MQTT_CLIENT_ID = "micropython-weather-demo"
MQTT_BROKER = "broker.mqttdashboard.com"
MQTT_USER = ""
MQTT_PASSWORD = ""
MQTT_TOPIC = "wokwi-weather"

sensor = dht.DHT22(Pin(15))

wlan = network.WLAN(network.STA_IF)
wlan.active(True)
wlan.connect("Wokwi-GUEST", "")
while not wlan.isconnected():
    time.sleep(0.2)
print("WiFi connected")

client = MQTTClient(
    MQTT_CLIENT_ID, MQTT_BROKER,
    user=MQTT_USER, password=MQTT_PASSWORD
)
client.connect()
print("MQTT connected")

while True:
    try:
        sensor.measure()
        message = ujson.dumps({
            "temp": sensor.temperature(),
            "humidity": sensor.humidity()
        })
```

```
        print("Publishing to", MQTT_TOPIC, ":", message)
        client.publish(MQTT_TOPIC, message)
    except Exception as error:
        print("Read/publish error:", error)
    time.sleep(5)
```

Wiring / compatibility note: DHT22 DATA/SDA → GP15, VCC → 3V3(OUT), GND → GND. The original code imports umqtt.simple; this module may need to be available on the Pico MicroPython firmware. If missing, install/provide the module or use the Arduino version. Public MQTT brokers may be unavailable or rate-limit clients.

Practical 8 — Interface a passive buzzer with Raspberry Pi Pico

Board / language: Raspberry Pi Pico — MicroPython

Based on the PDF's buzzer practical: GP15 and one-second ON/OFF intervals.

diagram.json

```
{
  "version": 1,
  "author": "Wokwi",
  "editor": "wokwi",
  "parts": [
    {
      "type": "board-pi-pico",
      "id": "pico",
      "top": 0,
      "left": 0,
      "attrs": {}
    },
    {
      "type": "wokwi-buzzer",
      "id": "buzz",
      "top": -20,
      "left": 180,
      "attrs": {}
    }
  ],
  "connections": [
    [
      "buzz:1",
      "pico:GP15",
      "green",
      []
    ],
    [
      "buzz:2",
      "pico:GND.1",
      "black",
      []
    ]
  ]
}
```

main.py

```
from machine import Pin, PWM
from time import sleep

# Passive buzzer positive pin -> GP15; negative pin -> GND
buzzer = PWM(Pin(15))
buzzer.freq(1000)

while True:
    buzzer.duty_u16(32768)  # buzzer ON
    sleep(1)
    buzzer.duty_u16(0)      # buzzer OFF
    sleep(1)
```

Wiring / compatibility note: The PDF uses picozero.Speaker(15); PWM is used here to make the tone generation explicit. The buzzer sounds for one second and is silent for one second.

Practical 9 — IoT-based DC motor speed control using potentiometer and servo

Board / language: Raspberry Pi Pico — MicroPython

Based on the PDF's PWM motor, servo and ADC potentiometer code.

diagram.json

```
{
  "version": 1,
  "author": "Wokwi",
  "editor": "wokwi",
  "parts": [
    {
      "type": "board-pi-pico",
      "id": "pico",
      "top": 0,
      "left": 0,
      "attrs": {}
    },
    {
      "type": "wokwi-potentiometer",
      "id": "pot1",
      "top": -30,
      "left": 180,
      "attrs": {
        "value": "0"
      }
    },
    {
      "type": "wokwi-servo",
      "id": "servo1",
      "top": 100,
      "left": 180,
      "attrs": {}
    },
    {
      "type": "wokwi-dc-motor",
      "id": "motor1",
      "top": 180,
      "left": 180,
      "attrs": {}
    }
  ],
  "connections": [
    [
      "pot1:VCC",
      "pico:3V3(OUT)",
      "red",
      []
    ],
    [
      "pot1:GND",
      "pico:GND.1",
      "black",
      []
    ],
    [
      "pot1:SIG",
      "pico:GP26",
      "green",
      []
    ],
    [
      "servo1:V+",
      "pico:VBUS",
      "red",
      []
    ],
    [
      "servo1:GND",
      "pico:GND.1",
      "black",
      []
    ],
    [
      "servo1:PWM",
      "pico:GP16",
      "orange",
      []
    ],
    [
      "motor1:IN1",
      "pico:GP15",
      "green",
      []
    ],
    [
      "motor1:IN2",
      "pico:GND.1",
      "black",
      []
```

```
    ]
  ]
}
```

main.py

```
from machine import Pin, PWM, ADC
import time

# DC motor PWM output (simulation demonstration)
dc_motor = PWM(Pin(15))
dc_motor.freq(1000)

# Servo motor PWM output
servo = PWM(Pin(16))
servo.freq(50)

# Potentiometer signal -> GP26 / ADC0
pot = ADC(26)

def set_servo_angle(angle):
    # Approximate 0.5 ms to 2.5 ms pulse at 50 Hz
    duty = int((angle / 180) * 6554 + 1638)
    servo.duty_u16(duty)

angle = 0
direction = 1

while True:
    pot_val = pot.read_u16()
    dc_motor.duty_u16(pot_val)

    speed_delay = (65535 - pot_val) / 1000000 + 0.001

    if pot_val > 1000:
        angle += direction
        if angle >= 180:
            angle = 180
            direction = -1
        elif angle <= 0:
            angle = 0
            direction = 1
        set_servo_angle(angle)
        speed_percent = int(pot_val * 100 / 65535)
        print("DC Speed: {}% | Servo Speed: {}%".format(
            speed_percent, speed_percent))
    else:
        print("Motors stopped")

    time.sleep(speed_delay)
```

Wiring / compatibility note: Potentiometer SIG → GP26, VCC → 3V3(OUT), GND → GND; servo signal → GP16. The supplied practical's servo pulse mapping and ADC approach are retained with boundary handling. A real DC motor must use a motor driver and separate suitable supply; do not power it directly from a GPIO pin. The Wokwi DC motor's pin model can vary; treat it as a visual/simulation aid and verify the diagram in the editor.

Practical 10 — IoT data logging to ThingSpeak using LDR

Board / language: Raspberry Pi Pico — MicroPython simulation adaptation

The PDF's original is BeagleBone Black + Adafruit_BBIO + LDR. Wokwi does not provide a BeagleBone Black board, so the diagram and code below adapt the same objective to Pico.

diagram.json

```
{
  "version": 1,
  "author": "Wokwi",
  "editor": "wokwi",
  "parts": [
    {
      "type": "board-pi-pico",
      "id": "pico",
      "top": 0,
      "left": 0,
      "attrs": {}
    },
    {
      "type": "wokwi-photoresistor-sensor",
      "id": "ldr1",
      "top": -30,
      "left": 180,
      "attrs": {}
    }
  ],
  "connections": [
    [
      "ldr1:VCC",
      "pico:3V3(OUT)",
      "red",
      []
    ],
    [
      "ldr1:GND",
      "pico:GND.1",
      "black",
      []
    ],
    [
      "ldr1:AO",
      "pico:GP26",
      "green",
      []
    ]
  ]
}
```

main.py

```
# Practical 10: LDR data logging to ThingSpeak
# Wokwi adaptation: Raspberry Pi Pico + LDR (original PDF uses BeagleBone Black).
import network
import time
import urequests
from machine import ADC

# On Pico, GP26 is ADC0. LDR AO -> GP26.
ldr = ADC(26)
WRITE_API_KEY = "YOUR_WRITE_API_KEY"
URL = "https://api.thingspeak.com/update"

wlan = network.WLAN(network.STA_IF)
wlan.active(True)
wlan.connect("Wokwi-GUEST", "")
while not wlan.isconnected():
    time.sleep(0.5)
print("WiFi connected")

while True:
    try:
        reading = ldr.read_u16()
        voltage = (reading / 65535) * 3.3
        print("ADC Reading:", reading)
        print("Voltage:", round(voltage, 4), "V")

        if WRITE_API_KEY != "YOUR_WRITE_API_KEY":
            response = urequests.get(
                URL + "?api_key=" + WRITE_API_KEY +
                "&field1=" + str(round(voltage, 4))
            )
            print("ThingSpeak response:", response.text)
            response.close()
        else:
            print("Set your ThingSpeak Write API Key to upload data.")
    except Exception as error:
        print("Logging error:", error)

    time.sleep(20)
```

Wiring / compatibility note: Create a ThingSpeak channel with Field 1 enabled and replace YOUR_WRITE_API_KEY. The original BeagleBone Black code uses Adafruit_BBIO.ADC and a 1.8 V ADC; those APIs do not work on Pico, so this adaptation uses Pico ADC on GP26 and scales to 3.3 V. ThingSpeak updates should be spaced by at least 15 seconds.